

6-Step Python Metaprogramming Exercises

What you will explore

Complete these exercises sequentially to review dynamic introspection, attribute access customization, class factories, and metaclass enforcement.

Exercise 1: Dynamic Metadata Introspection

Objective: Access dunder attributes like `__name__`, `__bases__`, `__mro__`, and `__dict__` to audit class structure dynamically.

Instructions: Write a function `inspect_class(cls)` that returns a summary dictionary of metadata for any class.

Code Template:

```
def inspect_class(cls):
    """
    Returns a dictionary with keys:
    'name', 'bases', 'mro', and 'attributes'.
    """
    # TODO: Populate and return the metadata summary dictionary
    pass

# Verification Code
class Parent:
    category = "base"

class Child(Parent):
    value = 100
    def run(self): pass

meta = inspect_class(Child)
print("Class Name:", meta.get("name"))
print("User Attributes:", sorted(meta.get("attributes", [])))
```

Expected Shell Output:

```
Class Name: Child
User Attributes: ['run', 'value']
```

Exercise 2: Dynamic Configuration Wrapper

Objective: Turn a dictionary data source into an attribute-based interface using `__getattr__`.

Instructions: Create a `ReadOnlyConfig` class that takes a dictionary and permits accessing nested keys via dot notation, raising `AttributeError` for missing attributes.

Code Template:

```
class ReadOnlyConfig:
    def __init__(self, data):
        object.__setattr__(self, "_data", dict(data))

    def __getattr__(self, name):
        # TODO: Implement dynamic lookup and wrap nested dicts
        pass

    def __setattr__(self, name, value):
        raise AttributeError("ReadOnlyConfig is immutable.")
```

Expected Shell Output:

```
Debug state: True
DB Host: localhost
```

Exercise 3: Strict Attribute Validation

Objective: Intercept attribute assignment to validate or normalise values.

Instructions: Build a StrictEmployee class that checks that 'salary' cannot be negative, and 'name' must be a non-empty string, raising TypeError or ValueError as appropriate.

Code Template:

```
class StrictEmployee:
    def __setattr__(self, name, value):
        # TODO: Add validations and delegate assignment using object.__setattr__
        pass
```

Expected Shell Output:

```
Employee registered: Alice, Salary: 75000.0
Pristine name validation error: Employee name cannot be empty
Pristine salary validation error: Salary cannot be negative
```

Exercise 4: Generating a Class with type()

Objective: Dynamically create custom class architectures programmatically.

Instructions: Write a function class_factory(class_name, fields) that uses type() to construct a new class on the fly with those fields initialized via __init__.

Code Template:

```
def class_factory(class_name, fields):
    # TODO: Implement dynamic __init__ and construct class via type()
    pass
```

Expected Shell Output:

```
Class Name: Point3D
Coordinates: 10 20 30
```

Exercise 5: Dynamic API Method Generator

Objective: Attach dynamic helper methods to classes programmatically based on a configuration list of strings.

Instructions: Write a function `generate_api_client(endpoints)` that dynamically binds helper methods named `get_`, each executing `mock_get(endpoint)` internally.

Code Template:

```
class ApiClient:
    def __init__(self, base_url):
        self.base_url = base_url

    def mock_get(self, endpoint):
        return f"GET request succeeded for {self.base_url}/{endpoint}"

def generate_api_client(endpoints):
    # TODO: Construct and populate class namespace with helper methods
    pass
```

Expected Shell Output:

```
Users API: GET request succeeded for https://api.system.internal/users
Orders API: GET request succeeded for https://api.system.internal/orders
```

Exercise 6: Metaclass Interface Verifier

Objective: Enforce structural layout rules and docstring presence at class compilation time.

Instructions: Construct a metaclass `InterfaceVerifier` that ensures any subclass of `BaseComponent` defines a non-empty `__doc__` docstring and implements a `validate()` method.

Code Template:

```
class InterfaceVerifier(type):
    def __new__(mcls, name, bases, namespace):
        # TODO: Enforce structural rules before constructing class
        pass
```

Expected Shell Output:

DatabaseComponent instantiated successfully.
Docstring check failed correctly:
Class 'BrokenDoc' must define a non-empty docstring.
Method check failed correctly:
Class 'BrokenMethod' must implement a 'validate(self)' method.